LoCED × MaxText v0.2.5 — Colab TPU validation

This notebook validates the **LoCED MaxText port** on a Google Colab TPU runtime:

- creates a **Python 3.12** environment and installs **MaxText v0.2.5** with the TPU extras;
- checks that JAX sees the TPU devices and uses the **real MaxText utilities** (not the CPU stubs);
- verifies the locked architecture: **4 physical layers, 8 visits, 4 FULL + REUSE/REUSE/REINDEX/REINDEX**;
- runs the **15 integration tests** (exact parity, MaxText Muon, sharding, checkpoint/resume);
- runs a short **MaxText Muon training benchmark**;
- verifies **Orbax checkpoint save → restore → resume** on the TPU.

Colab checklist

1. **Runtime → Change runtime type → TPU**, then save.
2. Internet is on by default. No local setup, files, or datasets are needed; the notebook clones `https://github.com/offx-zinth/loced.git` at `master`.
3. Run the cells in order. The first install takes roughly 5–15 minutes.
4. Keep the tab open for the whole run (~20–30 minutes).

What "pass" means here

- Integration tests pass with `LOCED_REQUIRE_REAL_MAXTEXT_UTILS=1` — so MaxText's real mesh, LR schedule, Muon optimizer, and Orbax path were exercised.
- Parity uses `LOCED_PARITY_TOL=1e-5` on TPU. The default CPU run is exact (`0.0`); TPU fusion can introduce tiny last-bit differences.
- Free Colab currently exposes **TPU v5e-1** (one chip). That validates the TPU backend and all model/optimizer/checkpoint paths, but not 8-device data parallelism. Use the Kaggle `TpuV5E8` notebook for the multi-device mesh.
- This is still the **tiny** config (549,456 parameters, sequence length 64). It validates correctness/plumbing on TPU, not large-scale throughput.

0. Configuration — repository, paths, accelerator

The notebook runs all heavy work in a separate Python 3.12 venv. That avoids Kaggle's kernel Python version and keeps MaxText's pinned JAX stack isolated.

```
# ======================================================================
# LoCED x MaxText v0.2.5 - Colab TPU validation
# ======================================================================
import json
import os
import shutil
import subprocess
import sys
import textwrap
import time
from pathlib import Path

# ----------------------------------------------------------------------
# User knobs
# ----------------------------------------------------------------------
ACCELERATOR = os.environ.get("LOCED_ACCELERATOR", "tpu")  # TPU valid
MAXTEXT_REF = "maxtext-v0.2.5"
MAXTEXT_REPO = "https://github.com/AI-Hypercomputer/maxtext.git"
MAXTEXT_EXTRAS = "cuda12" if ACCELERATOR == "gpu" else "tpu"

# The notebook clones this repo if LoCED is not already present.
LOCED_REPO_URL = os.environ.get(
    "LOCED_REPO_URL", "https://github.com/offx-zinth/loced.git"
)
LOCED_REPO_REF = os.environ.get("LOCED_REPO_REF", "master")

WORK = Path("/kaggle/working") if Path("/kaggle/working").is_dir() el
WORK.mkdir(parents=True, exist_ok=True)
MAXTEXT_DIR = WORK / "maxtext-v0.2.5"
VENV_DIR = WORK / "loced-venv"
PY = VENV_DIR / "bin" / "python"
SCRIPTS = WORK / "loced_scripts"
RUNS = WORK / "loced_runs"
CACHE = WORK / "jax_cache"
for _p in (SCRIPTS, RUNS, CACHE):
    _p.mkdir(parents=True, exist_ok=True)


def _loced_parent(root: Path):
    # Return the directory to put on PYTHONPATH for `import loced`.
    if (root / "loced" / "maxtext" / "train.py").is_file():
        return root
    # offx-zinth/loced is the package contents at the repo root.
    if root.name == "loced" and (root / "maxtext" / "train.py").is_fi
        return root.parent
    return None


def _find_loced_root():
    candidates = []
```

```
        if os.environ.get("LOCED_ROOT"):
            candidates.append(Path(os.environ["LOCED_ROOT"]))
        candidates += [
            WORK,
            WORK / "loced",
            Path.cwd(),
            Path.cwd() / "loced",
            Path("/kaggle/input"),
        ]
        for root in candidates:
            found = _loced_parent(root)
            if found is not None:
                return found
        kaggle_input = Path("/kaggle/input")
        if kaggle_input.is_dir():
            for p in kaggle_input.glob("**/maxtext/train.py"):
                repo = p.parents[1]
                if repo.name == "loced" or (
                    (repo / "attention.py").is_file()
                    and (repo / "config.py").is_file()
                ):
                    return repo.parent
            for p in kaggle_input.glob("**/loced/maxtext/train.py"):
                return p.parents[2]
        return None


    # Keep a reused Kaggle working directory in sync with the branch unde
    _target = WORK / "loced"
    if LOCED_REPO_URL and (_target / ".git").is_dir():
        _env = os.environ.copy()
        _env["GIT_TERMINAL_PROMPT"] = "0"
        print(f"updating existing checkout {_target} ({LOCED_REPO_REF})")
        subprocess.run(
            ["git", "-C", str(_target), "fetch", "--depth", "1",
             "origin", LOCED_REPO_REF],
            check=True, env=_env,
        )
        subprocess.run(
            ["git", "-C", str(_target), "reset", "--hard", "FETCH_HEAD"],
            check=True, env=_env,
        )

    LOCED_ROOT = _find_loced_root()
    if LOCED_ROOT is None and LOCED_REPO_URL:
        target = WORK / "loced"
        if not (target / ".git").is_dir():
            if target.is_dir() and any(target.iterdir()):
                print(f"{target} exists without .git; treating it as the
            else:
                env = os.environ.copy()
                env["GIT_TERMINAL_PROMPT"] = "0"
                print(f"cloning {LOCED_REPO_URL} ({LOCED_REPO_REF}) -> {t
                subprocess.run(
                    ["git", "clone", "--depth", "1", "--branch", LOCED_RE
```

```
                    LOCED_REPO_URL, str(target)],
                   check=True, env=env,
               )
    LOCED_ROOT = _find_loced_root()

print(f"ACCELERATOR : {ACCELERATOR}")
print(f"WORK        : {WORK}")
print(f"LOCED_ROOT  : {LOCED_ROOT}")
print(f"MAXTEXT_DIR : {MAXTEXT_DIR}")
print(f"VENV        : {VENV_DIR}")
if LOCED_ROOT is None:
    raise RuntimeError(textwrap.dedent('''
        LoCED source not found.
        - Upload the repository folder (containing `loced/`) to
          /kaggle/working/loced, or as a Kaggle Dataset under
          /kaggle/input/...
        - Or set LOCED_REPO_URL (and LOCED_REPO_REF) in this cell to a
          that contains `loced/`.
    '''))
print("nvidia-smi:")
if shutil.which("nvidia-smi"):
    subprocess.run(["nvidia-smi", "-L"], check=False)
else:
    print("nvidia-smi not found (CPU-only notebook kernel?)")
```

```
updating existing checkout /content/loced (master)
ACCELERATOR : tpu
WORK        : /content
LOCED_ROOT  : /content
MAXTEXT_DIR : /content/maxtext-v0.2.5
VENV        : /content/loced-venv
nvidia-smi:
nvidia-smi not found (CPU-only notebook kernel?)
```

1. Python 3.12 + MaxText v0.2.5 (TPU)

MaxText v0.2.5 requires Python >= 3.12, so the notebook creates its own venv with `uv` even if the Kaggle kernel is older. The install uses MaxText's tested dependency set:

```
uv pip install -e .[tpu] --resolution=lowest
```

Set `LOCED_SKIP_INSTALL=1` only when re-running against an already prepared venv.

```
SKIP_INSTALL = os.environ.get("LOCED_SKIP_INSTALL", "0") == "1"

if SKIP_INSTALL:
    print("LOCED_SKIP_INSTALL=1: using existing venv/MaxText checkout
else:
    if shutil.which("uv") is None:
```

```
            print("installing uv ...")
            subprocess.run([sys.executable, "-m", "pip", "install", "-q",
        UV = shutil.which("uv")
        if UV is None:
            raise RuntimeError("uv is not on PATH after pip install")

        if not PY.exists():
            print("creating Python 3.12 venv ...")
            subprocess.run(
                [UV, "venv", "--python", "3.12", "--seed", str(VENV_DIR)]
                check=True,
            )
        print(subprocess.check_output([str(PY), "--version"], text=True).

        if not (MAXTEXT_DIR / "pyproject.toml").is_file():
            print(f"cloning MaxText {MAXTEXT_REF} ...")
            subprocess.run(
                ["git", "clone", "--depth", "1", "--branch", MAXTEXT_REF,
                 MAXTEXT_REPO, str(MAXTEXT_DIR)],
                check=True,
            )
        else:
            print("MaxText checkout already present")

        print(f"installing maxtext[{MAXTEXT_EXTRAS}] (resolution=lowest)
        subprocess.run(
            [UV, "pip", "install", "--python", str(PY), "--link-mode=copy
             "-e", f".[{MAXTEXT_EXTRAS}]", "--resolution=lowest"],
            cwd=str(MAXTEXT_DIR), check=True,
        )
        subprocess.run(
            [UV, "pip", "install", "--python", str(PY), "--link-mode=copy
             "pytest", "pytest-timeout"],
            check=True,
        )
        print("dependency check:")
        subprocess.run([UV, "pip", "check", "--python", str(PY)], check=F

    print(subprocess.check_output(
        [str(PY), "-c",
         "import sys, maxtext; print(sys.version); print(maxtext.__file__
        text=True,
    ))
```

```
Python 3.12.3
MaxText checkout already present
installing maxtext[tpu] (resolution=lowest) ...
dependency check:
3.12.3 (main, Aug 31 2026, 10:18:26) [GCC 13.3.0]
/content/maxtext-v0.2.5/src/maxtext/__init__.py
```

2. TPU + real-MaxText sanity check

This is deliberately strict: the run fails if JAX is not on the TPU, or if MaxText's heavy utilities cannot be imported. A CPU-stub fallback would make the rest of the notebook meaningless on Kaggle.

```
BASE_ENV = os.environ.copy()
BASE_ENV.update({
    "PYTHONPATH": f"{MAXTEXT_DIR / 'src'}:{LOCED_ROOT}",
    "JAX_COMPILATION_CACHE_DIR": str(CACHE),
    "XLA_PYTHON_CLIENT_MEM_FRACTION": "0.90",
    "XLA_PYTHON_CLIENT_PREALLOCATE": "true",
    "TOKENIZERS_PARALLELISM": "false",
    "HF_HUB_DISABLE_PROGRESS_BARS": "1",
    "PYTHONDONTWRITEBYTECODE": "1",
    "TF_CPP_MIN_LOG_LEVEL": "2",
})

# jax-cuda12 loads the pip-installed NVIDIA libraries with ctypes.  I
# they are under site-packages/nvidia/*/lib, which is not on the defa
# path, so cuSPARSE/cuDNN can otherwise be reported as "not found".
_nvidia_code = (
    "import site; from pathlib import Path; "
    "print(':'.join(str(p) for r in site.getsitepackages() "
    "for p in sorted((Path(r) / 'nvidia').glob('*/lib')) if p.is_dir(
)
_nvidia_libs = subprocess.check_output([str(PY), "-c", _nvidia_code],
if _nvidia_libs:
    BASE_ENV["LD_LIBRARY_PATH"] = ":".join(
        x for x in (_nvidia_libs, os.environ.get("LD_LIBRARY_PATH", "
    )
print("NVIDIA lib dirs:", len(_nvidia_libs.split(":")) if _nvidia_lib


def run_script(name, code, timeout=3600, extra_env=None, check=True,
    path = SCRIPTS / name
    path.write_text(textwrap.dedent(code))
    env = BASE_ENV.copy()
    if extra_env:
        env.update({str(k): str(v) for k, v in extra_env.items()})
    t0 = time.time()
    proc = subprocess.run(
        [str(PY), str(path)],
        cwd=str(cwd or LOCED_ROOT),
        env=env,
        text=True,
        capture_output=True,
        timeout=timeout,
    )
    elapsed = time.time() - t0
    print(proc.stdout)
    if proc.stderr.strip():
        print("[stderr]\n" + proc.stderr[-12000:])
    print(f"[{name}] exit={proc.returncode} elapsed={elapsed:.1f}s")
    if check and proc.returncode != 0:
```

```
            raise RuntimeError(f"{name} failed (exit {proc.returncode})")
    return proc


gpu_check = r'''
import importlib.metadata
import os
import shutil
import subprocess
import sys

import jax
import maxtext

print("python  :", sys.version.split()[0])
print("jax     :", jax.__version__)
print("maxtext :", importlib.metadata.version("maxtext"))
print("backend :", jax.default_backend())
print("devices :", jax.devices())

expected = os.environ["LOCED_EXPECTED_BACKEND"]
assert jax.default_backend() == expected, (
    f"Expected {expected}, got {jax.default_backend()}. "
    "Set the Kaggle accelerator and restart the session."
)
assert len(jax.devices()) >= 1
for d in jax.devices():
    stats = d.memory_stats() or {}
    print(f"  {d}: {d.device_kind} bytes_limit={stats.get('bytes_limi

# The GPU/TPU run must exercise the real MaxText utilities, not CPU s
import maxtext.utils.maxtext_utils  # noqa: F401
import maxtext.utils.model_creation_utils  # noqa: F401
print("real MaxText utilities: OK")
if shutil.which("nvidia-smi"):
    subprocess.run(["nvidia-smi"], check=False)
'''
run_script(
    "gpu_check.py", gpu_check,
    extra_env={"LOCED_EXPECTED_BACKEND": ACCELERATOR},
)
```

```
NVIDIA lib dirs: 0
python  : 3.12.3
jax     : 0.11.1
maxtext : 0.2.5
backend : tpu
devices : [TpuDevice(id=0, process_index=0, coords=(0,0,0), core_on_ch
  TPU_0(process=0,(0,0,0,0)): TPU v5 lite bytes_limit=16909336064
real MaxText utilities: OK

[stderr]
/content/loced-venv/lib/python3.12/site-packages/jax/_src/cloud_tpu_in
  warnings.warn(
[transformers] PyTorch was not found. Models won't be available and on
[transformers] DeepseekV32Config got `key=rope_scaling` in kwargs but
```

```
[transformers] DeepseekV4Config got `key=rope_scaling` in kwargs but h

[gpu_check.py] exit=0 elapsed=13.7s
CompletedProcess(args=['/content/loced-venv/bin/python',
'/content/loced_scripts/gpu_check.py'], returncode=0, stdout='python
: 3.12.3\njax     : 0.11.1\nmaxtext : 0.2.5\nbackend : tpu\ndevices :
[TpuDevice(id=0, process_index=0, coords=(0,0,0), core_on_chip=0)]\n
TPU_0(process=0,(0,0,0,0)): TPU v5 lite bytes_limit=16909336064\nreal
MaxText utilities: OK\n', stderr='/content/loced-
venv/lib/python3.12/site-packages/jax/_src/cloud_tpu_init.py:90:
UserWarning: Transparent hugepages are not enabled. TPU runtime
startup and shutdown time should be significantly improved on TPU v5e
and newer. If not already set, you may need to enable transparent
hugepages in your VM image (sudo sh -c "echo always >
/sys/kernel/mm/transparent_hugepage/enabled")\n
warnings.warn(\n[transformers] PyTorch was not found. Models won\'t be
available and only tokenizers, configuration and file/data utilities
can be used.\n[transformers] DeepseekV32Config got `key=rope_scaling`
in kwargs but hasn\'t set it as attribute. For RoPE standardization
you need to set `self.rope_parameters` in model\'s config.
\n[transformers] DeepseekV4Config got `key=rope_scaling` in kwargs but
hasn\'t set it as attribute. For RoPE standardization you need to set
`self.rope_parameters` in model\'s config. \n')
```

3. Architecture + sharding on the TPU

Builds the NNX model with MaxText's real mesh and logical-axis rules. This cell fails if the 4-layer / 8-visit schedule or the compressed-attention fields have drifted.

```
arch_code = r'''
import json
import os

import jax
from maxtext.configs import pyconfig

from loced.maxtext.config import LoCEDMaxTextConfig, loced_config_fro
from loced.maxtext.train import create_model_and_mesh

accelerator = os.environ["LOCED_ACCELERATOR"]
n = len(jax.devices())
if accelerator == "gpu":
    mesh_overrides = [f"ici_fsdp_parallelism={n}", "ici_data_parallel
else:
    mesh_overrides = ["ici_fsdp_parallelism=1", f"ici_data_parallelis

config = pyconfig.initialize(
    ["loced-arch", "loced/maxtext/configs/loced_tiny.yml", *mesh_over
     "steps=2", "per_device_batch_size=2", "enable_checkpointing=fals
     "log_config=false", "skip_jax_distributed_system=true"],
    config_class=LoCEDMaxTextConfig,
)
cfg = loced_config_from_maxtext(config)
```

```
model, mesh = create_model_and_mesh(config)

print("params           :", f"{model.count_params():,}")
print("physical layers  :", cfg.num_layers)
print("visits           :", cfg.num_visits)
print("encode           :", cfg.encode_layers)
print("decode           :", cfg.decode_layers)
print("pool/window/top_k:", cfg.pool_m, cfg.window, cfg.top_k)
print("indexer          :", cfg.indexer_heads, "heads x", cfg.indexer_
print("experts          :", cfg.num_experts, "top", cfg.top_k_experts
      "shared", cfg.num_shared_experts)
print("mesh axes        :", mesh.axis_names)
print("mesh shape       :", mesh.shape)

assert cfg.num_layers == 4
assert cfg.num_visits == 8
assert cfg.encode_layers == ("full",) * 4
assert cfg.decode_layers == ("full",) * 4 + (
    "reuse", "reuse", "reindex", "reindex")
assert cfg.pool_m >= 1 and cfg.window >= 0 and cfg.top_k >= 0

for name, param in [
    ("token_embedding", model.token_embedding),
    ("layer0.W_q", model.cells["0"].attn.W_q),
    ("layer0.experts.wi_0", model.cells["0"].moe.routed_moe.wi_0),
    ("layer0.norm_a", model.cells["0"].norm_a),
]:
    print(f"{name:22s}", param.get_value().sharding)

report = {
    "backend": jax.default_backend(),
    "devices": n,
    "params": model.count_params(),
    "num_layers": cfg.num_layers,
    "num_visits": cfg.num_visits,
    "encode": list(cfg.encode_layers),
    "decode": list(cfg.decode_layers),
    "pool_m": cfg.pool_m,
    "window": cfg.window,
    "top_k": cfg.top_k,
    "indexer_heads": cfg.indexer_heads,
    "indexer_dim": cfg.indexer_dim,
    "num_experts": cfg.num_experts,
    "top_k_experts": cfg.top_k_experts,
}
with open(os.environ["LOCED_ARCH_REPORT"], "w") as f:
    json.dump(report, f, indent=2)
print("architecture report written")
'''
run_script(
    "architecture.py", arch_code,
    extra_env={
        "LOCED_ACCELERATOR": ACCELERATOR,
        "LOCED_ARCH_REPORT": str(WORK / "arch_report.json"),
```

```
    },
)
```

```
params           : 549,456
physical layers  : 4
visits           : 8
encode           : ('full', 'full', 'full', 'full')
decode           : ('full', 'full', 'full', 'full', 'reuse', 'reuse',
pool/window/top_k: 2 16 16
indexer          : 2 heads x 32
experts          : 4 top 2 shared 1
mesh axes        : ('diloco', 'data', 'stage', 'fsdp', 'fsdp_transpos
mesh shape       : OrderedDict({'diloco': 1, 'data': 1, 'stage': 1, '
token_embedding        NamedSharding(mesh=Mesh('diloco': 1, 'data': 1
layer0.W_q             NamedSharding(mesh=Mesh('diloco': 1, 'data': 1
layer0.experts.wi_0    NamedSharding(mesh=Mesh('diloco': 1, 'data': 1
layer0.norm_a          NamedSharding(mesh=Mesh('diloco': 1, 'data': 1
architecture report written

[stderr]
/content/loced-venv/lib/python3.12/site-packages/jax/_src/cloud_tpu_i
  warnings.warn(
[transformers] PyTorch was not found. Models won't be available and o
[transformers] DeepseekV32Config got `key=rope_scaling` in kwargs but
[transformers] DeepseekV4Config got `key=rope_scaling` in kwargs but
WARNING:absl:base_output_directory is not provided; Using local direc
WARNING:absl:tokenizer_path not found in HF_IDS in maxtext/src/maxtex

[architecture.py] exit=0 elapsed=16.6s
CompletedProcess(args=['/content/loced-venv/bin/python',
'/content/loced_scripts/architecture.py'], returncode=0,
stdout="params           : 549,456\nphysical layers  : 4\nvisits
: 8\nencode           : ('full', 'full', 'full', 'full')\ndecode
: ('full', 'full', 'full', 'full', 'reuse', 'reuse', 'reindex',
'reindex')\npool/window/top_k: 2 16 16\nindexer          : 2 heads x
32\nexperts          : 4 top 2 shared 1\nmesh axes        :
('diloco', 'data', 'stage', 'fsdp', 'fsdp_transpose', 'context',
'context_usp_ulysses', 'context_autoregressive', 'tensor',
'tensor_sequence', 'expert', 'autoregressive')\nmesh shape       :
OrderedDict({'diloco': 1, 'data': 1, 'stage': 1, 'fsdp': 1,
'fsdp_transpose': 1, 'context': 1, 'context_usp_ulysses': 1,
'context_autoregressive': 1, 'tensor': 1, 'tensor_sequence': 1,
'expert': 1, 'autoregressive': 1})\ntoken_embedding
NamedSharding(mesh=Mesh('diloco': 1, 'data': 1, 'stage': 1, 'fsdp':
1, 'fsdp_transpose': 1, 'context': 1, 'context_usp_ulysses': 1,
'context_autoregressive': 1, 'tensor': 1, 'tensor_sequence': 1,
'expert': 1, 'autoregressive': 1, axis_types=(Auto, Auto, Auto,
Auto, Auto, Auto, Auto, Auto, Auto, Auto, Auto, Auto)), spec=P(None,
None), memory_kind=device)\nlayer0.W_q
NamedSharding(mesh=Mesh('diloco': 1, 'data': 1, 'stage': 1, 'fsdp':
1, 'fsdp_transpose': 1, 'context': 1, 'context_usp_ulysses': 1,
'context_autoregressive': 1, 'tensor': 1, 'tensor_sequence': 1,
'expert': 1, 'autoregressive': 1, axis_types=(Auto, Auto, Auto,
Auto, Auto, Auto, Auto, Auto, Auto, Auto, Auto, Auto)), spec=P(None,
None), memory_kind=device)\nlayer0.experts.wi_0
NamedSharding(mesh=Mesh('diloco': 1, 'data': 1, 'stage': 1, 'fsdp':
1, 'fsdp_transpose': 1, 'context': 1, 'context_usp_ulysses': 1,
'context_autoregressive': 1, 'tensor': 1, 'tensor_sequence': 1,
'expert': 1, 'autoregressive': 1, axis_types=(Auto, Auto, Auto,
Auto, Auto, Auto, Auto, Auto, Auto, Auto, Auto, Auto)), spec=P(None,
```

4. Full MaxText integration suite

Runs `loced/maxtext/test_integration.py` with:

- `LOCED_REQUIRE_REAL_MAXTEXT_UTILS=1` — no silent CPU stubs;
- `LOCED_PARITY_TOL=1e-5` — TPU parity tolerance (CPU default is exact `0.0`).

This covers weights, loss/logits, gradients, prefill/decode, MaxText Muon, sharding classification, config, checkpoint round-trip, and trainer resume.

```
test_env = BASE_ENV.copy()
test_env.update({
    "LOCED_PARITY_TOL": "1e-5",
    "LOCED_REQUIRE_REAL_MAXTEXT_UTILS": "1",
    "LOCED_ACCELERATOR": ACCELERATOR,
})
cmd = [
    str(PY), "-m", "pytest", "-q",
    "loced/maxtext/test_integration.py",
    "-x", "--durations=10", "--timeout=1800", "-p", "no:cacheprovider
]
t0 = time.time()
proc = subprocess.run(
    cmd, cwd=str(LOCED_ROOT), env=test_env,
    text=True, capture_output=True, timeout=5400,
)
print(proc.stdout)
if proc.stderr.strip():
    print("[stderr]\n" + proc.stderr[-12000:])
print(f"pytest exit={proc.returncode} elapsed={time.time() - t0:.1f}s
if proc.returncode != 0:
    raise RuntimeError("MaxText integration tests failed")
print("MaxText integration tests: PASS")
```

```
.............
============================== warnings summary ====================
loced-venv/lib/python3.12/site-packages/jax/_src/cloud_tpu_init.py:90
  /content/loced-venv/lib/python3.12/site-packages/jax/_src/cloud_tpu_
    warnings.warn(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.htm
============================== slowest 10 durations ====================
12.77s call     loced/maxtext/test_integration.py::test_gradients_are_
6.70s call     loced/maxtext/test_integration.py::test_forward_and_los
6.16s call     loced/maxtext/test_integration.py::test_prefill_and_dec
5.84s call     loced/maxtext/test_integration.py::test_trainer_resumes
2.74s call     loced/maxtext/test_integration.py::test_maxtext_muon_on
2.61s call     loced/maxtext/test_integration.py::test_orbax_checkpoin
2.22s setup    loced/maxtext/test_integration.py::test_nnx_parameter_i
0.78s call     loced/maxtext/test_integration.py::test_trainer_rejects
```

```
0.34s call     loced/maxtext/test_integration.py::test_shipped_yaml_in
0.02s setup    loced/maxtext/test_integration.py::test_forward_and_los
15 passed, 1 warning in 53.52s

pytest exit=0 elapsed=57.5s
MaxText integration tests: PASS
```

5. MaxText Muon training benchmark

Two warmup steps compile the jitted train step; the next 20 steps are timed. This is a **global synthetic batch** on the tiny config, so read it as a plumbing/throughput sanity check, not a model-scale benchmark.

```
bench_code = r'''
import json
import os
import time

import jax
import numpy as np
from maxtext.configs import pyconfig

from loced.maxtext.config import LoCEDMaxTextConfig, loced_config_fro
from loced.maxtext.train import (
    create_model_and_mesh, create_optimizer, synthetic_batches, train_
)

accelerator = os.environ["LOCED_ACCELERATOR"]
n = len(jax.devices())
if accelerator == "gpu":
    mesh_overrides = [f"ici_fsdp_parallelism={n}", "ici_data_parallel
else:
    mesh_overrides = ["ici_fsdp_parallelism=1", f"ici_data_parallelis

warmup = int(os.environ.get("LOCED_BENCH_WARMUP", "2"))
steps = int(os.environ.get("LOCED_BENCH_STEPS", "20"))
batch = int(os.environ.get("LOCED_BENCH_BATCH", "4"))
config = pyconfig.initialize(
    ["loced-bench", "loced/maxtext/configs/loced_tiny.yml", *mesh_ove
     f"steps={warmup + steps}", f"per_device_batch_size={batch}",
     "enable_checkpointing=false", "log_config=false",
     "skip_jax_distributed_system=true"],
    config_class=LoCEDMaxTextConfig,
)
cfg = loced_config_from_maxtext(config)
model, mesh = create_model_and_mesh(config)
schedule, optimizer = create_optimizer(config, model, mesh)
print("params", f"{model.count_params():,}", "devices", n,
      "batch", batch, "seq", cfg.max_seq_len)

batches = list(synthetic_batches(
```

```
    cfg.vocab_size, cfg.max_seq_len, batch, warmup + steps, seed=0))
before = np.asarray(model.cells["0"].attn.W_q.get_value()).copy()

for i in range(warmup):
    loss, _ = train_step(model, optimizer, batches[i])
    jax.block_until_ready(loss)
print("warmup done")

losses = []
t0 = time.perf_counter()
for i in range(warmup, warmup + steps):
    loss, metrics = train_step(model, optimizer, batches[i])
    losses.append(float(loss))
    jax.block_until_ready(loss)
t1 = time.perf_counter()

elapsed = t1 - t0
tokens = steps * batch * cfg.max_seq_len
after = np.asarray(model.cells["0"].attn.W_q.get_value())
print("loss first/last:", losses[0], losses[-1])
print("loss delta     :", losses[-1] - losses[0])
print("optimizer step :", int(optimizer.step.get_value()))
print(f"step time      : {elapsed / steps * 1000:.1f} ms")
print(f"tokens/sec     : {tokens / elapsed:,.0f} (global synthetic ba
print("param changed  :", not np.array_equal(before, after))
for d in jax.devices():
    stats = d.memory_stats() or {}
    print(f"memory {d}: peak={stats.get('peak_bytes_in_use')} "
          f"limit={stats.get('bytes_limit')}")

assert np.isfinite(losses).all()
assert int(optimizer.step.get_value()) == warmup + steps
assert not np.array_equal(before, after)

report = {
    "backend": jax.default_backend(),
    "devices": n,
    "batch": batch,
    "seq_len": cfg.max_seq_len,
    "params": model.count_params(),
    "warmup_steps": warmup,
    "timed_steps": steps,
    "loss_first": losses[0],
    "loss_last": losses[-1],
    "ms_per_step": elapsed / steps * 1000,
    "tokens_per_sec": tokens / elapsed,
    "optimizer_step": int(optimizer.step.get_value()),
    "param_changed": bool(not np.array_equal(before, after)),
}
with open(os.environ["LOCED_TRAIN_REPORT"], "w") as f:
    json.dump(report, f, indent=2)
'''
run_script(
    "benchmark.py", bench_code, timeout=3600,
    extra_env={
```

```
            "LOCED_ACCELERATOR": ACCELERATOR,
            "LOCED_TRAIN_REPORT": str(WORK / "train_report.json"),
        },
    )
```

```
params 549,456 devices 1 batch 4 seq 64
warmup done
loss first/last: 4.654140472412109 4.295656204223633
loss delta     : -0.35848426818847656
optimizer step : 22
step time      : 7.1 ms
tokens/sec     : 36,212 (global synthetic batch)
param changed  : True
memory TPU_0(process=0,(0,0,0,0)): peak=41703936 limit=16909336064

[stderr]
/content/loced-venv/lib/python3.12/site-packages/jax/_src/cloud_tpu_in
  warnings.warn(
[transformers] PyTorch was not found. Models won't be available and on
[transformers] DeepseekV32Config got `key=rope_scaling` in kwargs but
[transformers] DeepseekV4Config got `key=rope_scaling` in kwargs but h
WARNING:absl:base_output_directory is not provided; Using local direct
WARNING:absl:tokenizer_path not found in HF_IDS in maxtext/src/maxtext

[benchmark.py] exit=0 elapsed=19.1s
CompletedProcess(args=['/content/loced-venv/bin/python',
'/content/loced_scripts/benchmark.py'], returncode=0, stdout='params
549,456 devices 1 batch 4 seq 64\nwarmup done\nloss first/last:
4.654140472412109 4.295656204223633\nloss delta     :
-0.35848426818847656\noptimizer step : 22\nstep time      : 7.1
ms\ntokens/sec     : 36,212 (global synthetic batch)\nparam changed  :
True\nmemory TPU_0(process=0,(0,0,0,0)): peak=41703936
limit=16909336064\n', stderr='/content/loced-venv/lib/python3.12/site-
packages/jax/_src/cloud_tpu_init.py:90: UserWarning: Transparent
hugepages are not enabled. TPU runtime startup and shutdown time
should be significantly improved on TPU v5e and newer. If not already
set, you may need to enable transparent hugepages in your VM image
(sudo sh -c "echo always >
/sys/kernel/mm/transparent_hugepage/enabled")\n
warnings.warn(\n[transformers] PyTorch was not found. Models won\'t be
available and only tokenizers, configuration and file/data utilities
can be used.\n[transformers] DeepseekV32Config got `key=rope_scaling`
in kwargs but hasn\'t set it as attribute. For RoPE standardization
you need to set `self.rope_parameters` in model\'s config.
\n[transformers] DeepseekV4Config got `key=rope_scaling` in kwargs but
hasn\'t set it as attribute. For RoPE standardization you need to set
`self.rope_parameters` in model\'s config.
\nWARNING:absl:base_output_directory is not provided; Using local
directory called maxtext_output\nWARNING:absl:tokenizer_path not found
in HF_IDS in maxtext/src/maxtext/utils/globals.py.          Using the
default src/maxtext/assets/tokenizers/tokenizer.llama2 instead.
Please pass tokenizer_path in your command if this is not
intended.\n')
```

6. Orbax checkpoint save → restore → resume

Checks that the TPU run can save model + optimizer state, restore it into a fresh model, and continue from the saved optimizer step.

```
ckpt_code = r'''
import json
import os
import shutil
from pathlib import Path

import jax
import numpy as np
from flax import nnx
from maxtext.configs import pyconfig

from loced.maxtext.config import LoCEDMaxTextConfig, loced_config_fro
from loced.maxtext.train import (
    checkpoint_path, create_model_and_mesh, create_optimizer,
    load_checkpoint, save_checkpoint, synthetic_batches, train_step,
)

accelerator = os.environ["LOCED_ACCELERATOR"]
n = len(jax.devices())
if accelerator == "gpu":
    mesh_overrides = [f"ici_fsdp_parallelism={n}", "ici_data_parallel
else:
    mesh_overrides = ["ici_fsdp_parallelism=1", f"ici_data_parallelis

base = os.environ["LOCED_CKPT_BASE"]
config = pyconfig.initialize(
    ["loced-ckpt", "loced/maxtext/configs/loced_tiny.yml", *mesh_over
     "steps=3", "per_device_batch_size=2", "enable_checkpointing=true
     "checkpoint_period=1", f"base_output_directory={base}",
     "run_name=loced-kaggle-ckpt", "log_config=false",
     "skip_jax_distributed_system=true"],
    config_class=LoCEDMaxTextConfig,
)
cfg = loced_config_from_maxtext(config)
ckpt = checkpoint_path(config)
shutil.rmtree(ckpt, ignore_errors=True)
print("checkpoint dir:", ckpt)

model, mesh = create_model_and_mesh(config)
schedule, optimizer = create_optimizer(config, model, mesh)
tokens = next(iter(synthetic_batches(
    cfg.vocab_size, cfg.max_seq_len, 2, 1, seed=0)))
for _ in range(2):
    loss, _ = train_step(model, optimizer, tokens)
    jax.block_until_ready(loss)
assert int(optimizer.step.get_value()) == 2
save_checkpoint(ckpt, model, optimizer)
print("saved at step", int(optimizer.step.get_value()))

model2, mesh2 = create_model_and_mesh(config)
```

```
_, optimizer2 = create_optimizer(config, model2, mesh2)
restored = load_checkpoint(ckpt, model2, optimizer2, mesh2)
print("restored step:", restored)
assert restored == 2
assert int(optimizer2.step.get_value()) == 2

a = jax.tree.leaves(nnx.to_pure_dict(nnx.state(model)))
b = jax.tree.leaves(nnx.to_pure_dict(nnx.state(model2)))
assert all(np.array_equal(np.asarray(x), np.asarray(y))
           for x, y in zip(a, b))

loss, _ = train_step(model2, optimizer2, tokens)
jax.block_until_ready(loss)
assert int(optimizer2.step.get_value()) == 3
print("resumed to step", int(optimizer2.step.get_value()))

files = [p for p in Path(ckpt).rglob("*") if p.is_file()]
size = sum(p.stat().st_size for p in files)
print("checkpoint files:", len(files), "bytes:", size)

report = {
    "checkpoint_dir": ckpt,
    "restored_step": restored,
    "resumed_step": int(optimizer2.step.get_value()),
    "files": len(files),
    "bytes": size,
}
with open(os.environ["LOCED_CKPT_REPORT"], "w") as f:
    json.dump(report, f, indent=2)
'''
run_script(
    "checkpoint.py", ckpt_code, timeout=3600,
    extra_env={
        "LOCED_ACCELERATOR": ACCELERATOR,
        "LOCED_CKPT_BASE": str(RUNS),
        "LOCED_CKPT_REPORT": str(WORK / "ckpt_report.json"),
    },
)
```

```
checkpoint dir: /content/loced_runs/loced-kaggle-ckpt/checkpoints/
saved at step 2
restored step: 2
resumed to step 3
checkpoint files: 14 bytes: 4385506

[stderr]
/content/loced-venv/lib/python3.12/site-packages/jax/_src/cloud_tpu_in
  warnings.warn(
[transformers] PyTorch was not found. Models won't be available and on
[transformers] DeepseekV32Config got `key=rope_scaling` in kwargs but
[transformers] DeepseekV4Config got `key=rope_scaling` in kwargs but h
WARNING:absl:tokenizer_path not found in HF_IDS in maxtext/src/maxtext

[checkpoint.py] exit=0 elapsed=21.0s
CompletedProcess(args=['/content/loced-venv/bin/python',
'/content/loced_scripts/checkpoint.py'], returncode=0,
```

```
stdout='checkpoint dir: /content/loced_runs/loced-kaggle-
ckpt/checkpoints/\nsaved at step 2\nrestored step: 2\nresumed to step
3\ncheckpoint files: 14 bytes: 4385506\n', stderr='/content/loced-
venv/lib/python3.12/site-packages/jax/_src/cloud_tpu_init.py:90:
UserWarning: Transparent hugepages are not enabled. TPU runtime
startup and shutdown time should be significantly improved on TPU v5e
and newer. If not already set, you may need to enable transparent
hugepages in your VM image (sudo sh -c "echo always >
/sys/kernel/mm/transparent_hugepage/enabled")\n
warnings.warn(\n[transformers] PyTorch was not found. Models won\'t be
available and only tokenizers, configuration and file/data utilities
can be used.\n[transformers] DeepseekV32Config got `key=rope_scaling`
in kwargs but hasn\'t set it as attribute. For RoPE standardization
you need to set `self.rope_parameters` in model\'s config.
\n[transformers] DeepseekV4Config got `key=rope_scaling` in kwargs but
hasn\'t set it as attribute. For RoPE standardization you need to set
`self.rope_parameters` in model\'s config.
\nWARNING:absl:tokenizer_path not found in HF_IDS in
maxtext/src/maxtext/utils/globals.py.          Using the default
src/maxtext/assets/tokenizers/tokenizer.llama2 instead.
Please pass tokenizer_path in your command if this is not
intended.\n')
```

7. Summary

If every cell above completed, the Colab TPU run is green. The Kaggle T4 GPU and Kaggle TPU v5e-8 notebooks are in the same repository.

```
reports = {}
for name in ("arch_report.json", "train_report.json", "ckpt_report.js
    p = WORK / name
    if p.is_file():
        reports[name] = json.loads(p.read_text())

print(json.dumps(reports, indent=2))

summary = {
    "accelerator": ACCELERATOR,
    "backend": reports.get("arch_report.json", {}).get("backend"),
    "devices": reports.get("arch_report.json", {}).get("devices"),
    "params": reports.get("arch_report.json", {}).get("params"),
    "integration_tests": "PASS",
    "parity_tolerance": 1e-5,
    "train": reports.get("train_report.json"),
    "checkpoint": reports.get("ckpt_report.json"),
}
report_path = WORK / "colab_tpu_report.json"
report_path.write_text(json.dumps(summary, indent=2))
print("Wrote", report_path)
print("Colab TPU validation complete.")
```

```
{
  "arch_report.json": {
```

```
        "backend": "tpu",
        "devices": 1,
        "params": 549456,
        "num_layers": 4,
        "num_visits": 8,
        "encode": [
          "full",
          "full",
          "full",
          "full"
        ],
        "decode": [
          "full",
          "full",
          "full",
          "full",
          "reuse",
          "reuse",
          "reindex",
          "reindex"
        ],
        "pool_m": 2,
        "window": 16,
        "top_k": 16,
        "indexer_heads": 2,
        "indexer_dim": 32,
        "num_experts": 4,
        "top_k_experts": 2
      },
      "train_report.json": {
        "backend": "tpu",
        "devices": 1,
        "batch": 4,
        "seq_len": 64,
        "params": 549456,
        "warmup_steps": 2,
        "timed_steps": 20,
        "loss_first": 4.654140472412109,
        "loss_last": 4.295656204223633,
        "ms_per_step": 7.069516499996098,
        "tokens_per_sec": 36211.81165644656,
        "optimizer_step": 22,
        "param_changed": true
      },
      "ckpt_report.json": {
        "checkpoint_dir": "/content/loced_runs/loced-kaggle-ckpt/checkpoin
        "restored_step": 2,
        "resumed_step": 3,
        "files": 14,
        "bytes": 4385506
      }
    }
    Wrote /content/colab_tpu_report.json
    Colab TPU validation complete.
```